

Position: AI Data Engineer

Objective

Build a simple customer-facing AI assistant using a **Retrieval-Augmented Generation (RAG)** approach, with an **agentic backend implemented using LangGraph**.

The assistant should answer questions based on content from a VodafoneZiggo web page.

1. High-level Goal

The assistant helps Ziggo customers with questions about products and services on ziggo.nl, especially around internet, TV and related options. Examples:

- What TV packages does Ziggo offer?
- What is Ziggo GO and how does it work?

2. Data & RAG Setup

2.1 Source page

- Select web page, for example:
<https://www.ziggo.nl/internet>.

2.2 Scraping and text extraction

- Use a simple Python script (for example: requests + BeautifulSoup) to:
 - Download the HTML.
 - Extract the main content text (focus on product/service information).

2.3 Cleaning and chunking

- Clean the extracted text in a straightforward way:
 - Remove obvious boilerplate such as navigation, footer, cookie banners.
- Split the cleaned text into chunks suitable for retrieval, for example:
 - Fixed-size chunks (e.g. by number of characters or tokens), or

- Section-based chunks (e.g. per heading).

2.4 Embeddings and vector store

- Once you have your cleaned chunks of text, pick an embedding model to represent them numerically — for example an OpenAI embedding model, a Hugging Face model, or another public option that fits your needs. For every chunk, create an embedding and then store the result together with the original chunk text and any useful metadata, such as the section title or content type if needed.
- Store these embeddings in a local vector store so you can perform similarity search later. You can use a lightweight solution like Chroma, OpenSearch, or a comparable alternative, depending on your preferences and environment. In your code comments, briefly explain why you selected that embedding model and vector store, touching on aspects such as quality, cost, latency, and ease of use.

3. LangGraph Agentic Workflow

Implement the backend orchestration using **LangGraph**.

3.1 Workflow design

- Define a workflow (graph) that is triggered when a user sends a question.
- Design the nodes and data flow yourself. For each node, make clear (in code or README):

- What input it receives

(for example: question, retrieved documents, intermediate state).

- What processing it performs

(for example: retrieval, answer generation, validation).

- What output it returns to the next node.

3.2 Required behaviour

Ensure that the LangGraph workflow covers at least:

1. Accepting the user question as input.
2. Retrieving relevant chunks from the vector store.
3. Calling an LLM or similar model with:

- a. The question.
 - b. The retrieved context.
 - c. Simple system instructions (tone, style, safety).
4. Producing a final answer that:
 - a. Uses the retrieved content.
 - b. Is suitable as a customer-facing response.

Add simple handling for low-confidence or empty retrieval results, for example:

- Return a clear “cannot answer based on this content” message, or
- Follow an alternative path in the graph (e.g. reformulate query).

4. API & Containerisation

4.1 FastAPI application

- Build a minimal **FastAPI** application exposing an endpoint, for example:
 - POST /ask
- This endpoint should:
 - Receive a user question (e.g. in JSON).
 - Call the LangGraph workflow.
 - Return the final answer as JSON.

4.2 Docker and Docker Compose

- Create a **Dockerfile** for the application.
- Create a **docker-compose.yml** that starts:
 - The FastAPI service.
 - The vector store service (if it runs as a separate process).
- After starting with Docker Compose, it should be possible to:
 - Send a request to the API endpoint.
 - Receive an answer based on the scraped page.

4.3 Documentation in code

- Use inline comments to explain key implementation choices, especially:
 - How the LangGraph workflow is structured.
 - How the workflow integrates with the API and vector store.
 - How you handle errors or missing data.

5. Architecture & AWS Representation

5.1 Local architecture

Create a simple architecture diagram showing the local flow, for example:

User → FastAPI / API → LangGraph workflow → Vector store → Answer → User

Show the main nodes/steps in the workflow.

5.2 AWS view

Briefly explain:

- How the main components would scale.
- Any basic security considerations.

6. Deliverables

Please provide:

- **Source code (Python)** with inline comments.
- **LangGraph workflow definition(s)** showing the agentic orchestration.
- **Dockerfile** and **docker-compose.yml** for running the solution end-to-end.
- **README** describing:
 - How to run the solution locally with Docker Compose.
 - Which models, libraries, and vector store you chose, and why.
 - The design of your LangGraph workflow (main nodes and data flow).
- **Architecture diagram(s)** for:
 - The local setup.
 - The AWS representation.

7. Hints & Clarifications

- Scraping and text cleaning can remain simple; focus on:
 - RAG setup, and
 - LangGraph workflow design.
- You may use OpenAI, Hugging Face, or similar providers for embeddings and LLMs.
- Make sure end-to-end:
 - A user can call the API endpoint.
 - The system returns an answer based on the scraped page.
- The assignment is expected to take approximately **3–5 hours** to complete.